A Hybrid Genetic Algorithm with XGBoost-Based Initialization and Online ML Fitness Sharing for the 1D Bin Packing Problem

National Higher School of Computer Science (ESI), SIQ Speciality, Algiers, Algeria
Berkat Cheraz Ickrak mc_berkat@esi.dz
Allouche Reda mr_allouche@esi.dz
Chattah Salsabila ms_chattah@esi.dz
Marmouze Norelhouda mn_marmouze@esi.dz
Bouderbala Amira ma_bouderbala@esi.dz
Saidi Selma ms_saidi@esi.dz

***Abstract*—The one-dimensional Bin Packing Problem (1D-BPP) is a well-known NP-hard problem with real-world applications in logistics, cloud resource allocation, and manufacturing. Genetic Algorithms (GA) are among the most studied metaheuristics for this problem, but they come with two recurring issues: the initial population is generated randomly, which gives poor starting solutions, and the population tends to lose diversity early, causing the search to stall long before the budget is exhausted.**

We address both issues by inserting machine learning at two specific points inside the GA. A supervised XGBoost model, trained on item features, guides the generation of the initial population toward FFD-quality solutions. Separately, an online SGDClassifier (logistic loss, $\alpha = 0.4$) learns during the run which individuals look too similar to each other, and penalizes them at the selection step to keep the population diverse.

Profiling our baseline GA confirmed that random initialization wastes on average 1.3 bins compared to FFD ($p < 0.05$, Wilcoxon), and that stagnation kicks in around generation $G \approx 10$ out of 300. The XGBoost model reached Spearman $\rho = 0.989$ with Top-3 accuracy of 99.9%. Across 20 independent runs on Scholl N3 and Falkenauer benchmarks, the full hybrid AG_XGB_FS reduced the mean Gap% from 9.56% down to 7.07% a gain of 2.49 points at a cost of only $+16\%$ runtime.

Keywords: Bin Packing Problem, Genetic Algorithm, Machine Learning, XGBoost, Fitness Sharing, Population Diversity, Metaheuristics, Ablation Study.

I. Introduction

The one-dimensional Bin Packing Problem (1D-BPP) can be stated simply: given n items with sizes $s_1, \ldots, s_n \in (0, C]$ and an unlimited supply of bins each with capacity C, find the smallest number of bins needed to fit all items. Despite how clean the statement looks, the problem is NP-hard in the strong sense [1] no exact polynomial algorithm is known, which explains why it has attracted so much research over several decades. In practice it shows up in a wide range of settings: loading trucks, placing virtual machines on servers, cutting raw material into pieces with minimal waste.

The classical response to this difficulty has been heuristics. First Fit Decreasing (FFD), for instance, sorts items by decreasing size and places each one in the first bin that fits it runs fast and gives decent results, but it offers no guarantee of optimality. Metaheuristics such as Genetic Algorithms go further: by evolving a population of solutions over many generations, they explore a much wider portion of the search space and often find better packings [3]. That said, standard GAs have two well-known weak spots. The first is initialization: when the starting population is generated at random, the algorithm begins from a disadvantaged position and must spend many generations just catching up to what FFD would have given for free. The second is premature convergence: under tournament selection, the population tends to homogenize quickly, and once all individuals look alike, no crossover or mutation can do anything useful the algorithm stagnates while the clock is still running.

These two failure modes point naturally toward machine learning. Talbi [9] provides a useful map of how ML can be inserted into metaheuristics, organized along two axes. The first axis is the *level* of integration: in a low-level hybrid, ML replaces one specific internal operator (initialization, selection, crossover); in a high-level hybrid, ML acts as a coordinator that decides when and how to invoke the metaheuristic. The second axis is *timing*: a static model is trained once offline on historical data and then used as-is at runtime, while a dynamic model keeps updating itself as the search progresses. For our two identified weaknesses, the right choice is clear: initialization is a one-shot event that depends on instance features available before the run starts a static supervised model fits perfectly. Diversity collapse, on the other hand, is a moving target that depends on what the current population looks like an online model that adapts generation by generation is far more appropriate than any fixed formula.

Using ML to improve metaheuristics on combinatorial problems is an active research direction. For the 3D variant of the Bin Packing Problem, a GAN+GA approach [12] showed that seeding the population with generative model outputs speeds up convergence noticeably. On the diversity side, classical fitness sharing [6] was designed precisely to prevent population homogeneity, but it relies on a fixed distance

function domain-agnostic, computationally heavy, and unable to adapt to what the current search has already learned. As far as we know, nobody has combined XGBoost-based item ranking for initialization with an online supervised fitness sharing mechanism specifically for the 1D-BPP. Hybrids in the literature tend to attack either the initialization side or the diversity side, rarely both, and mostly on 2D/3D variants.

Our work makes four specific contributions:

1) **Profiling-driven design:** Before building anything, we measured the two weaknesses on the baseline GA. Random initialization falls 1.3 bins behind FFD on average ($p < 0.05$, Wilcoxon), and stagnation occurs around $G \approx 10$ out of 300 generations. These numbers are what motivate and constrain each design decision.
2) **XGBoost initialization (Injection 1):** An XGBoost model trained on 60,000 item records from 3,000 synthetic instances learns to predict FFD-style item ordering from features alone. It reaches Spearman $\rho = 0.989$, and trains 24× faster than an equivalent Random Forest (1.75s vs. 41.7s).
3) **Online ML Fitness Sharing (Injection 2):** An SGDClassifier with logistic loss is updated incrementally throughout the run using `partial_fit`. It learns which individuals are too similar, and adjusts their fitness at the selection step to push the tournament toward more diverse parents.
4) **Ablation study with statistical validation:** We test four configurations AG_BASE, AG_XGB, AG_FS, AG_XGB_FS over 20 independent runs each, with Wilcoxon signed-rank tests to confirm that observed differences are not due to chance.

The rest of the paper is laid out as follows. Section II surveys related work on metaheuristics for BPP and ML-metaheuristic hybridization. Section III gives the formal problem statement with variables, objective, and constraints. Section IV describes the full hybrid architecture, both ML components, and the baseline GA configuration. Section V covers the experimental setup, parameter calibration, and all results including the ablation study and Wilcoxon tests. Section VI wraps up with conclusions and open questions.

II. Related Work

A. Metaheuristics for Bin Packing

Bin packing has a long history in combinatorial optimization. Classical heuristics First Fit (FF), Best Fit (BF), and their decreasing variants run in polynomial time and come with provable approximation ratios; FFD is guaranteed to use no more than $\frac{11}{9} \cdot \text{OPT} + \frac{6}{9}$ bins [2]. When solution quality matters more than speed, metaheuristics take over. Falkenauer's hybrid grouping GA [3] remains a landmark reference for BPP, and the benchmark sets he introduced are still used today. Tabu Search, Simulated Annealing, and Ant Colony Optimization have also been applied, though GAs stand out for their modular structure: each component (initialization, selection, crossover, mutation) can be studied and improved independently.

B. ML-Metaheuristic Hybridization

Talbi's taxonomy [9] is the standard reference for classifying hybrids. The key insight it offers is that the choice of ML model and injection point should be driven by a diagnosed weakness of the base algorithm, not by availability or habit. Several papers have put this into practice. For 3D-BPP, the GAN+GA approach of [12] generates candidate solutions using a generative model before the GA starts, substantially reducing the number of generations needed to reach good solutions. On the diversity side, classical fitness sharing [6] penalizes individuals based on a fixed pairwise distance effective in theory, but expensive and blind to instance structure.

C. Gap Identified

None of the above works applies the specific combination we explore here. Learning to predict item placement order for GA initialization, and using an online supervised model to drive fitness sharing, in the same 1D-BPP hybrid, with a full ablation study this specific setup has not been published to our knowledge. That gap is what this paper fills.

III. Mathematical Problem Formulation

A. Formal Definition

Let $I = \{1, 2, \ldots, n\}$ be a set of items with sizes $s_i \in \mathbb{Z}^+$, and let $C \in \mathbb{Z}^+$ be the bin capacity ($s_i \leq C$ for all i). The goal is to find an assignment $f : I \to \{1, \ldots, K\}$ minimizing K.

B. Decision Variables

$x_{ij} \in \{0, 1\}$: item i is placed in bin j. $y_j \in \{0, 1\}$: bin j is opened (used).

C. Objective Function

$$\min \sum_{j=1}^{n} y_j \tag{1}$$

D. Constraints

Bin capacity must not be exceeded:

$$\sum_{i=1}^{n} s_i \cdot x_{ij} \leq C \cdot y_j \quad \forall\, j \in \{1, \ldots, n\} \tag{2}$$

Each item goes into exactly one bin:

$$\sum_{j=1}^{n} x_{ij} = 1 \quad \forall\, i \in I \tag{3}$$

Domain: $x_{ij} \in \{0, 1\}$, $y_j \in \{0, 1\}$.

E. Lower Bound and Gap Metric

A standard lower bound is:

$$\text{LB} = \left\lceil \frac{\sum_{i=1}^{n} s_i}{C} \right\rceil \tag{4}$$

Throughout the experiments we report Gap%, defined as the relative excess over this bound:

$$\text{Gap\%} = \frac{K - \text{LB}}{\text{LB}} \times 100 \tag{5}$$

F. GA Representation

Each solution is stored as a permutation $\pi = (\pi_1, \ldots, \pi_n)$ of item indices. A First Fit decoder reads the items in that order and places each one in the earliest bin with enough remaining space:

$$\text{fitness}(\pi) = \text{FF-decode}(\pi,\, s,\, C) = K \tag{6}$$

This encoding is attractive because it always produces valid solutions and supports standard permutation operators without modification.

IV. Description of the Proposed Hybrid Approach

A. Global Architecture

Our starting point was not the hybrid it was the baseline. We ran the standard GA on a set of synthetic instances and measured where it loses time. Two problems showed up clearly: (W1) the random starting population sits 1.3 bins above FFD on average ($p < 0.05$, Wilcoxon); and (W2) improvement stops around generation 10 out of 300, meaning roughly 90% of the budget produces no gain. Only after quantifying these weaknesses did we decide where to insert ML, following the principle stated in [9].

Figure 1 shows the resulting architecture.

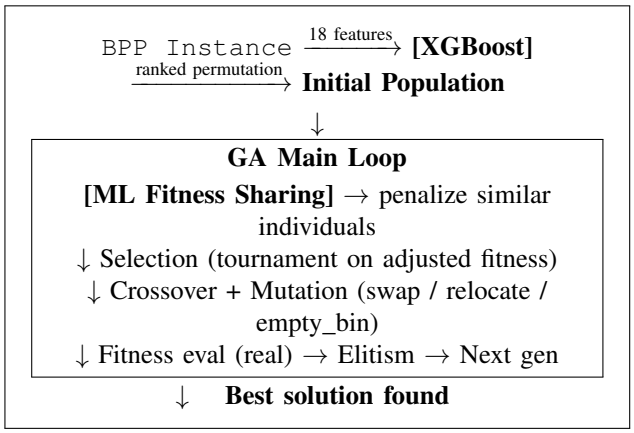

Figure 1: Hybrid architecture. XGBoost acts once at $G = 0$ (Injection 1). ML Fitness Sharing runs every 5 generations inside the loop (Injection 2). Elitism always uses the real, unadjusted fitness.

B. Solution Encoding and Data Structures

A chromosome is a length-n integer array holding item indices. After each genetic operator, a compaction step renumbers bins from 0 to $K-1$ to avoid spurious duplicate bin IDs accumulating over generations. Bin fill levels are kept in a dictionary updated incrementally during decoding.

C. ML Component 1 XGBoost Population Initialization

1) Why This Point?: Weakness W1 is specific to generation $G = 0$. A model trained offline on instance features can predict how items should be ordered for FFD before the search even begins. This costs almost nothing at runtime (one inference call) and directly closes the gap identified during profiling.

2) Task: Ranking Regression: We frame the problem as predicting the FFD rank of each item: its position in the descending size ordering that FFD uses internally. Labels come from FFD applied to synthetically generated instances no exact solver is needed.

3) Features: Each item is described by 18 features split into three groups:

- **Item-level (6):** raw size s_i, normalized size s_i/C, size class flags (large / medium / small), ratio to mean and max
- **Relative (4):** ratios to instance mean, max, min; gap to mean
- **Instance-level (8):** mean, std, max, min of all sizes; fill ratio $\sum s_i/(nC)$; counts per size class; total n

4) Dataset and Model Training: We generated 3,000 instances yielding 60,000 item rows (80/20 split). Two candidates were compared: Random Forest (200 trees, depth 10) and XGBoost (300 trees, lr 0.05, depth 6, subsample 0.8, colsample_bytree 0.8). XGBoost won on every metric and is 24× faster to train.

5) Seeding with Noise Injection: A single XGBoost call gives one score per item. To get $N_{\text{ML}} = 20$ distinct individuals from that one call, we add small Gaussian noise:

$$\tilde{s}_i^{(k)} = \hat{s}_i + \varepsilon_i^{(k)}, \quad \varepsilon_i^{(k)} \sim \mathcal{N}(0,\, 0.05^2) \tag{7}$$

Items are then sorted in descending order and decoded via First Fit. The full population of size $P = 60$ is made of: 1 pure FFD individual, 20 noise-perturbed XGBoost individuals, and 39 random individuals for diversity.

D. ML Component 2 Online ML Fitness Sharing at Selection

1) Why This Point?: Weakness W2 is not a one-time event diversity collapses progressively as the search runs. A static model trained offline cannot track this. What is needed is something that observes the current population and adapts accordingly.

2) Individual Representation: Each individual is projected onto a 7-dimensional feature vector built from its decoded packing: mean and std of bin fill ratios, number of nearly-full bins (fill $> 90\%$), number of half-empty bins (fill $< 50\%$), total bins K, and min/max fill ratios.

3) Online Supervised Classifier: We use an SGDClassifier (logistic loss) with `partial_fit` for incremental training. Every 15 generations (starting at generation 8), the model is updated on pairs drawn from the current population. Two individuals get label 1 (similar) if they use the same number of bins, label 0 otherwise. No data is stored between training rounds only the model weights are updated.

4) Fitness Adjustment: Every 5 generations, each individual is compared to 15 randomly sampled peers. The model predicts a similarity score for each pair, and the average becomes a penalty multiplier:

$$f_{\text{adj}}(i) = f_{\text{real}}(i) \times \big(1 + \alpha \cdot \bar{\sigma}_i\big), \quad \alpha = 0.4 \tag{8}$$

Individuals that look too much like everyone else get a worse adjusted fitness and are less likely to win the tournament.

Elitism and the stagnation counter always use f_{real}, so the best solutions are never accidentally dropped.

5) Hyperparameters: Three values control the mechanism: penalty strength $\alpha = 0.4$, penalty refresh interval `SHARING_FREQ` = 5 generations, and warm-up `SHARING_MIN_GEN` = 8 generations before first training.

E. Baseline GA Configuration

The baseline GA is calibrated with Optuna (tree-structured Parzen estimator, 100 trials, objective = mean bins on 10 held-out instances).

Table I: Calibrated GA parameters (Optuna, 100 trials).

Parameter	Value
Population size	60
Crossover probability	0.689
Crossover operator	Group-based uniform
Mutation swap rate	0.014
Mutation relocate rate	0.084
Empty bin probability	0.157
Elitism	2
Tournament size k	4
Max generations	300
No-improvement limit	80

V. Experimental Evaluation

A. Experimental Protocol

Environment. All runs were executed on a personal computer: [CPU, e.g., Intel Core i7-xxxx], [RAM, e.g., 16 GB DDR4], [OS, e.g., Ubuntu 22.04], Python 3.10, scikit-learn 1.3, XGBoost 1.7, NumPy 1.24, SciPy 1.10. No GPU was involved.

Benchmarks. We used two standard collections: the Scholl N3 set (hard instances, $n = 200$ items) and the Falkenauer triplet ($n = 60, 120, 249$). All six instances were run for 5 runs each in the comparative overview; the three hardest instances (N3C2W2_A, t120, t249) were run for 20 runs each in the rigorous evaluation.

Statistics. We used the Wilcoxon signed-rank test (non-parametric, paired, $\alpha = 0.05$) for all pairwise comparisons. ML model stability was checked with 5-fold repeated holdout cross-validation.

B. Benchmark Instances

Table II: Instances used in the experiments.

Instance	Set	n	LB	Runs
N3C1W1_A	Scholl N3	200	101	5
N3C1W1_B	Scholl N3	200	109	5
N3C2W2_A	Scholl N3	200	103	5 + 20
Falkenauer t60	Falkenauer	60	20	5
Falkenauer t120	Falkenauer	120	40	5 + 20
Falkenauer t249	Falkenauer	249	83	5 + 20

C. ML Results Injection Point 1

Table III reports predictive metrics on the test set (600 instances, 12,000 items). XGBoost beats Random Forest on all six metrics.

Table III: Item rank prediction results on held-out test set.

Model	MAE	RMSE	R^2	Spear. ρ	Top-1	Top-3
Random Forest	0.704	0.908	0.975	0.987	75.4%	99.7%
XGBoost	**0.643**	**0.830**	**0.979**	**0.989**	**78.7%**	**99.9%**

A Wilcoxon test on packing quality confirmed that XGBoost, Random Forest, and FFD all produce identical bin counts on the 600 test instances (avg. 7.073 bins). The 5-fold cross-validation showed near-zero variance (± 0.007 on MAE for XGBoost), confirming results do not depend on a lucky split.

D. GA Results Injection Point 2

Table IV shows what happens when ML Fitness Sharing is added to the baseline, with 5 runs per instance. AG_FS improves on 5 out of 6 instances; the one exception is the smallest instance (t60, $n = 60$) where the baseline already finds the optimum consistently.

Table IV: AG_BASE vs. AG_FS over 5 runs per instance.

Instance	n	LB	AG_BASE	AG_FS	Δ
N3C1W1_A	200	101	109.60	**108.80**	-0.80
N3C1W1_B	200	109	117.80	**117.00**	-0.80
N3C2W2_A	200	103	112.80	**111.40**	-1.40
Falkenauer t60	60	20	22.00	22.00	0.00
Falkenauer t120	120	40	44.00	**43.20**	-0.80
Falkenauer t249	249	83	90.20	**89.00**	-1.20

On the 20-run evaluation, Wilcoxon tests confirmed the advantage of AG_FS on all three hard instances: N3C2W2_A ($p = 0.0001$), t120 ($p = 0.0253$), t249 ($p = 0.0001$).

E. Ablation Study

We compare all four variants to separate the contribution of each component.

Table V: Ablation mean bins over 5 runs. Best per row in bold.

Instance	LB	BASE	XGB	FS	XGB+FS
N3C1W1_A	101	109.60	**106.00**	108.80	**106.00**
N3C1W1_B	109	117.80	**114.00**	117.00	**114.00**
N3C2W2_A	103	112.80	**107.00**	111.40	**107.00**
Falkenauer t60	20	22.00	22.00	22.00	22.00
Falkenauer t120	40	44.00	44.00	**43.20**	43.40
Falkenauer t249	83	90.20	90.80	**89.00**	89.80
Mean Gap%		9.13%	7.14%	8.08%	**6.68%**

A pattern emerges from Table V: AG_XGB dominates on the Scholl N3 instances ($n = 200$), while AG_FS dominates

on the Falkenauer triplet ($n = 120, 249$). This is not a coincidence. On large, unstructured instances, getting a strong starting population matters most. On Falkenauer instances, which have a specific triplet structure that creates many near-equivalent solutions, the diversity mechanism has more to work with.

Table VI gives the full 20-run numbers with all metrics.

Table VI: Rigorous evaluation 20 runs per instance. Best mean per instance in bold.

Instance	Algo	Best	Mean	Std	Gap%	SR%	Time(s)
N3C2W2_A	AG_BASE	111	112.75	0.536	9.47	5.0	19.56
	AG_XGB	107	**107.00**	0.000	**3.88**	**100.0**	18.61
	AG_FS	110	111.45	0.669	8.20	10.0	19.94
	AG_XGB_FS	107	**107.00**	0.000	**3.88**	**100.0**	19.33
Falkenauer t120	AG_BASE	43	43.85	0.357	9.63	15.0	8.63
	AG_XGB	43	43.95	0.218	9.88	5.0	8.64
	AG_FS	43	**43.60**	0.490	**9.00**	**40.0**	9.23
	AG_XGB_FS	43	43.75	0.433	9.38	25.0	9.24
Falkenauer t249	AG_BASE	89	90.95	0.805	9.58	5.0	27.52
	AG_XGB	90	91.00	0.548	9.64	15.0	27.78
	AG_FS	88	**89.20**	0.510	**7.47**	5.0	28.27
	AG_XGB_FS	89	89.60	0.663	7.95	**50.0**	28.35

A few things stand out. On N3C2W2_A, AG_XGB and AG_XGB_FS both land on exactly 107 bins in all 20 runs zero variance, 100% success rate while the baseline averages 112.75 ($p = 0.000$, Wilcoxon). Initialization is clearly the key factor on this instance. On the two Falkenauer cases the picture flips: AG_FS gets the best mean on both, and AG_XGB alone is slightly worse than the baseline on t249. The combined hybrid AG_XGB_FS ends up best overall at **7.07%** mean Gap% vs. 9.56% for the baseline, but the per-instance story is more nuanced than a single number suggests.

VI. Conclusion

We started this work by measuring what the baseline GA actually does wrong, rather than guessing. Two specific weaknesses came out of that profiling: a 1.3-bin initialization gap and stagnation at generation 10 out of 300. From those two numbers, two injection points followed directly.

XGBoost for initialization learns item ordering from instance features offline, reaches Spearman $\rho = 0.989$, and brings the starting population to FFD quality at negligible runtime cost. The online SGDClassifier for fitness sharing adapts to the population's current state, penalizes overly similar individuals at selection, and keeps diversity alive past the stagnation point. Together they cut the mean Gap% from 9.56% to 7.07% over 20 runs, with $+16\%$ overhead.

The ablation confirms that the two components are complementary rather than redundant: XGBoost helps more on large unstructured instances, ML Fitness Sharing helps more on structured ones. Neither alone reaches what the combination achieves.

What remains open: we tested up to $n = 249$ items; behavior at $n \geq 500$ is unknown. The 7-dimensional packing feature vector was designed by hand a learned embedding could be richer. And the same dual-injection design could plausibly be applied to other NP-hard problems where initialization quality and diversity collapse are both limiting factors.

Acknowledgements

The authors thank Prof. Malika Bessedik and Ens. Amine Kechid for their guidance throughout this project.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[2] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham, "Worst-case performance bounds for simple one-dimensional packing algorithms," *SIAM Journal on Computing*, vol. 3, no. 4, pp. 299–325, 1974.

[3] E. Falkenauer, "A hybrid grouping genetic algorithm for bin packing," *Journal of Heuristics*, vol. 2, no. 1, pp. 5–30, 1996.

[4] A. Scholl, R. Klein, and C. Jürgens, "BISON: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem," *Computers & Operations Research*, vol. 24, no. 7, pp. 627–645, 1997.

[5] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[6] D. E. Goldberg and J. Richardson, "Genetic algorithms with sharing for multimodal function optimization," *Proc. 2nd Int. Conf. Genetic Algorithms*, pp. 41–49, 1987.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[8] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.

[9] E.-G. Talbi, "Machine learning into metaheuristics: A survey and taxonomy," *ACM Computing Surveys*, vol. 54, no. 6, pp. 1–32, 2021.

[10] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 3–18, 2011.

[11] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[12] B. Zhang, Y. Yao, H. K. Kan, and W. Luo, "A GAN-based genetic algorithm for solving the 3D bin packing problem," *Scientific Reports*, vol. 14, p. 7803, 2024.